

IMPORTANT ALGORITHMS

QR METHOD

INPUT $n; a_1^{(1)}, \dots, a_n^{(1)}, b_2^{(1)}, \dots, b_n^{(1)}$; tolerance TOL ; maximum number of iterations M .

OUTPUT eigenvalues of A , or recommended splitting of A , or a message that the maximum number of iterations was exceeded.

Step 1 Set $k = 1$;
 $SHIFT = 0$. (Accumulated shift.)

Step 2 While $k \leq M$ do Steps 3–19.
 (Steps 3–7 test for success.)

Step 3 If $|b_n^{(k)}| \leq TOL$ then set $\lambda = a_n^{(k)} + SHIFT$;
 OUTPUT (λ);
 set $n = n - 1$.

Step 4 If $|b_2^{(k)}| \leq TOL$ then set $\lambda = a_1^{(k)} + SHIFT$;
 OUTPUT (λ);
 set $n = n - 1$;
 $a_1^{(k)} = a_2^{(k)}$;
 for $j = 2, \dots, n$
 set $a_j^{(k)} = a_{j+1}^{(k)}$;
 $b_j^{(k)} = b_{j+1}^{(k)}$.

Step 5 If $n = 0$ then
 STOP.

Step 6 If $n = 1$ then
 set $\lambda = a_1^{(k)} + SHIFT$;
 OUTPUT (λ);
 STOP.

Step 7 For $j = 3, \dots, n - 1$
 if $|b_j^{(k)}| \leq TOL$ then
 OUTPUT ('split into', $a_1^{(k)}, \dots, a_{j-1}^{(k)}, b_2^{(k)}, \dots, b_{j-1}^{(k)}$,
 'and',
 $a_j^{(k)}, \dots, a_n^{(k)}, b_{j+1}^{(k)}, \dots, b_n^{(k)}, SHIFT$);
 STOP.

Step 8 (*Compute shift.*)

Set $b = -(a_{n-1}^{(k)} + a_n^{(k)});$

$$c = a_n^{(k)} a_{n-1}^{(k)} - [b_n^{(k)}]^2;$$

$$d = (b^2 - 4c)^{1/2}.$$

Step 9 If $b > 0$ then set $\mu_1 = -2c/(b + d);$

$$\mu_2 = -(b + d)/2$$

else set $\mu_1 = (d - b)/2;$

$$\mu_2 = 2c/(d - b).$$

Step 10 If $n = 2$ then set $\lambda_1 = \mu_1 + SHIFT;$

$$\lambda_2 = \mu_2 + SHIFT;$$

OUTPUT $(\lambda_1, \lambda_2);$

STOP.

Step 11 Choose σ so that $|\sigma - a_n^{(k)}| = \min\{|\mu_1 - a_n^{(k)}|, |\mu_2 - a_n^{(k)}|\}.$

Step 12 (*Accumulate the shift.*)

Set $SHIFT = SHIFT + \sigma.$

Step 13 (*Perform shift.*)

For $j = 1, \dots, n$, set $d_j = a_j^{(k)} - \sigma.$

Step 14 (Steps 14 and 15 compute $R^{(k)}$.)

Set $x_1 = d_1$;

$y_1 = b_2$.

Step 15 For $j = 2, \dots, n$

$$\text{set } z_{j-1} = \left\{ x_{j-1}^2 + [b_j^{(k)}]^2 \right\}^{1/2};$$

$$c_j = \frac{x_{j-1}}{z_{j-1}};$$

$$\sigma_j = \frac{b_j^{(k)}}{z_{j-1}};$$

$$q_{j-1} = c_j y_{j-1} + s_j d_j;$$

$$x_j = -\sigma_j y_{j-1} + c_j d_j;$$

If $j \neq n$ then set $r_{j-1} = \sigma_j b_{j+1}^{(k)}$;

$$y_j = c_j b_{j+1}^{(k)}.$$

($A_j^{(k)} = P_j A_{j-1}^{(k)}$ has just been computed and $R^{(k)} = A_n^{(k)}$.)

Step 16 (Steps 16–18 compute $A^{(k+1)}$.)

Set $z_n = x_n$;

$$a_1^{(k+1)} = \sigma_2 q_1 + c_2 z_1;$$

$$b_2^{(k+1)} = \sigma_2 z_2.$$

Step 17 For $j = 2, 3, \dots, n-1$

$$\text{set } a_j^{(k+1)} = \sigma_{j+1} q_j + c_j c_{j+1} z_j;$$

$$b_{j+1}^{(k+1)} = \sigma_{j+1} z_{j+1}.$$

Step 18 Set $a_n^{(k+1)} = c_n z_n$.

Step 19 Set $k = k + 1$.

Step 20 OUTPUT ('Maximum number of iterations exceeded');

(The procedure was unsuccessful.)

STOP.

HOUSEHOLDER

To obtain a symmetric tridiagonal matrix $A^{(n-1)}$ similar to the symmetric matrix $A = A^{(1)}$, construct the following matrices $A^{(2)}, A^{(3)}, \dots, A^{(n-1)}$, where $A^{(k)} = (a_{ij}^{(k)})$ for each $k = 1, 2, \dots, n-1$:

INPUT dimension n ; matrix A .

OUTPUT $A^{(n-1)}$. (At each step, A can be overwritten.)

Step 1 For $k = 1, 2, \dots, n-2$ do Steps 2–14.

Step 2 Set

$$q = \sum_{j=k+1}^n \left(a_{jk}^{(k)}\right)^2.$$

Step 3 If $a_{k+1,k}^{(k)} = 0$ then set $\alpha = -q^{1/2}$

$$\text{else set } \alpha = -\frac{q^{1/2} a_{k+1,k}^{(k)}}{|a_{k+1,k}^{(k)}|}.$$

Step 4 Set $RSQ = \alpha^2 - \alpha a_{k+1,k}^{(k)}$. (Note: $RSQ = 2r^2$)

Step 5 Set $v_k = 0$; (Note: $v_1 = \dots = v_{k-1} = 0$, but are not needed.)

$$v_{k+1} = a_{k+1,k}^{(k)} - \alpha;$$

For $j = k+2, \dots, n$ set $v_j = a_{jk}^{(k)}$.

$$\left(\text{Note: } \mathbf{w} = \left(\frac{1}{\sqrt{2RSQ}}\right) \mathbf{v} = \frac{1}{2r} \mathbf{v}.\right)$$

Step 6 For $j = k, k+1, \dots, n$ set $u_j = \left(\frac{1}{RSQ}\right) \sum_{i=k+1}^n a_{ji}^{(k)} v_i$.

$$\left(\text{Note: } \mathbf{u} = \left(\frac{1}{RSQ} \right) A^{(k)} \mathbf{v} = \frac{1}{2r^2} A^{(k)} \mathbf{v} = \frac{1}{r} A^{(k)} \mathbf{w}. \right)$$

$$\text{Step 7} \quad \text{Set } PROD = \sum_{i=k+1}^n v_i u_i.$$

$$\left(\text{Note: } PROD = \mathbf{v}^t \mathbf{u} = \frac{1}{2r^2} \mathbf{v}^t A^{(k)} \mathbf{v}. \right)$$

$$\text{Step 8} \quad \text{For } j = k+1, k+2, \dots, n \text{ set } z_j = u_j - \left(\frac{PROD}{2RSQ} \right) v_j.$$

$$\begin{aligned} \left(\text{Note: } \mathbf{z} = \mathbf{u} - \frac{1}{2RSQ} \mathbf{v}^t \mathbf{u} \mathbf{v} = \mathbf{u} - \frac{1}{4r^2} \mathbf{v}^t \mathbf{u} \mathbf{v} \right. \\ \left. = \mathbf{u} - \mathbf{w} \mathbf{w}^t \mathbf{u} = \frac{1}{r} A^{(k)} \mathbf{w} - \mathbf{w} \mathbf{w}^t \frac{1}{r} A^{(k)} \mathbf{w}. \right) \end{aligned}$$

$$\text{Step 9} \quad \text{For } l = k+1, k+2, \dots, n-1 \text{ do Steps 10 and 11.}$$

$$\begin{aligned} (\text{Note: Compute } A^{(k+1)} &= A^{(k)} - \mathbf{v} \mathbf{z}^t - \mathbf{z} \mathbf{v}^t \\ &= (I - 2\mathbf{w} \mathbf{w}^t) A^{(k)} (I - 2\mathbf{w} \mathbf{w}^t).) \end{aligned}$$

$$\text{Step 10} \quad \text{For } j = l+1, \dots, n \text{ set}$$

$$a_{jl}^{(k+1)} = a_{jl}^{(k)} - v_l z_j - v_j z_l;$$

$$a_{lj}^{(k+1)} = a_{jl}^{(k+1)}.$$

$$\text{Step 11} \quad \text{Set } a_{ll}^{(k+1)} = a_{ll}^{(k)} - 2v_l z_l.$$

$$\text{Step 12} \quad \text{Set } a_{nn}^{(k+1)} = a_{nn}^{(k)} - 2v_n z_n.$$

$$\text{Step 13} \quad \text{For } j = k+2, \dots, n \text{ set } a_{kj}^{(k+1)} = a_{jk}^{(k+1)} = 0.$$

$$\text{Step 14} \quad \text{Set } a_{k+1,k}^{(k+1)} = a_{k+1,k}^{(k)} - v_{k+1} z_k;$$

$$a_{k,k+1}^{(k+1)} = a_{k+1,k}^{(k+1)}.$$

$$(\text{Note: The other elements of } A^{(k+1)} \text{ are the same as } A^{(k)}.)$$

$$\text{Step 15} \quad \text{OUTPUT } (A^{(n-1)});$$

(The process is complete. $A^{(n-1)}$ is symmetric, tridiagonal, and similar to A .)

STOP. ■

INVERSE POWER METHOD

To approximate an eigenvalue and an associated eigenvector of the $n \times n$ matrix A given a nonzero vector $\mathbf{x}$:

INPUT dimension n ; matrix A ; vector $\mathbf{x}$; tolerance TOL ; maximum number of iterations N .

OUTPUT approximate eigenvalue μ ; approximate eigenvector $\mathbf{x}$ (with $\|\mathbf{x}\|_\infty = 1$) or a message that the maximum number of iterations was exceeded.

Step 1 Set $q = \frac{\mathbf{x}^t A \mathbf{x}}{\mathbf{x}^t \mathbf{x}}$.

Step 2 Set $k = 1$.

Step 3 Find the smallest integer p with $1 \leq p \leq n$ and $|x_p| = \|\mathbf{x}\|_\infty$.

Step 4 Set $\mathbf{x} = \mathbf{x}/x_p$.

Step 5 While ($k \leq N$) do Steps 6–12.

Step 6 Solve the linear system $(A - qI)\mathbf{y} = \mathbf{x}$.

Step 7 If the system does not have a unique solution, then
OUTPUT (' q is an eigenvalue', q);
STOP.

Step 8 Set $\mu = y_p$.

Step 9 Find the smallest integer p with $1 \leq p \leq n$ and $|y_p| = \|\mathbf{y}\|_\infty$.

Step 10 Set $ERR = \|\mathbf{x} - (\mathbf{y}/y_p)\|_\infty$;
 $\mathbf{x} = \mathbf{y}/y_p$.

Step 11 If $ERR < TOL$ then set $\mu = (1/\mu) + q$;
OUTPUT ($\mu, \mathbf{x}$);
(The procedure was successful.)
STOP.

Step 12 Set $k = k + 1$.

Step 13 OUTPUT ('Maximum number of iterations exceeded');
(The procedure was unsuccessful.)
STOP.

POWER METHOD

To approximate the dominant eigenvalue and an associated eigenvector of the $n \times n$ matrix A given a nonzero vector $\mathbf{x}$:

INPUT dimension n ; matrix A ; vector $\mathbf{x}$; tolerance TOL ; maximum number of iterations N .

OUTPUT approximate eigenvalue μ ; approximate eigenvector $\mathbf{x}$ (with $\|\mathbf{x}\|_\infty = 1$) or a message that the maximum number of iterations was exceeded.

Step 1 Set $k = 1$.

Step 2 Find the smallest integer p with $1 \leq p \leq n$ and $|x_p| = \|\mathbf{x}\|_\infty$.

Step 3 Set $\mathbf{x} = \mathbf{x}/x_p$.

Step 4 While ($k \leq N$) do Steps 5–11.

Step 5 Set $\mathbf{y} = A\mathbf{x}$.

Step 6 Set $\mu = y_p$.

Step 7 Find the smallest integer p with $1 \leq p \leq n$ and $|y_p| = \|\mathbf{y}\|_\infty$.

Step 8 If $y_p = 0$ then OUTPUT ('Eigenvector', $\mathbf{x}$);
OUTPUT ('A has the eigenvalue 0, select a new vector $\mathbf{x}$ and restart');
STOP.

Step 9 Set $ERR = \|\mathbf{x} - (\mathbf{y}/y_p)\|_\infty$;
 $\mathbf{x} = \mathbf{y}/y_p$.

Step 10 If $ERR < TOL$ then OUTPUT ($\mu, \mathbf{x}$);
(The procedure was successful.)
STOP.

Step 11 Set $k = k + 1$.

Step 12 OUTPUT ('The maximum number of iterations exceeded');
(The procedure was unsuccessful.)
STOP.



CONJUGATE GRADIENT METHOD

INPUT the number of equations and unknowns n ; the entries a_{ij} , $1 \leq i, j \leq n$ of the matrix A ; the entries b_j , $1 \leq j \leq n$ of the vector $\mathbf{b}$; the entries γ_{ij} , $1 \leq i, j \leq n$ of the preconditioning matrix C^{-1} , the entries x_i , $1 \leq i \leq n$ of the initial approximation $\mathbf{x} = \mathbf{x}^{(0)}$, the maximum number of iterations N ; tolerance TOL .

OUTPUT the approximate solution $x_1, \dots, x_n$ and the residual $r_1, \dots, r_n$ or a message that the number of iterations was exceeded.

Step 1 Set $\mathbf{r} = \mathbf{b} - A\mathbf{x}$; (Compute $\mathbf{r}^{(0)}$.)
 $\mathbf{w} = C^{-1}\mathbf{r}$; (Note: $\mathbf{w} = \mathbf{w}^{(0)}$)
 $\mathbf{v} = C^{-t}\mathbf{w}$; (Note: $\mathbf{v} = \mathbf{v}^{(1)}$)
 $\alpha = \sum_{j=1}^n w_j^2$.

Step 2 Set $k = 1$.

Step 3 While ($k \leq N$) do Steps 4–7.

Step 4 If $\|\mathbf{v}\| < TOL$, then
 OUTPUT ('Solution vector'; $x_1, \dots, x_n$);
 OUTPUT ('with residual'; $r_1, \dots, r_n$);
 (The procedure was successful.)
 STOP

Step 5 Set $\mathbf{u} = A\mathbf{v}$; (Note: $\mathbf{u} = A\mathbf{v}^{(k)}$)
 $t = \frac{\alpha}{\sum_{j=1}^n v_j u_j}$; (Note: $t = t_k$)
 $\mathbf{x} = \mathbf{x} + t\mathbf{v}$; (Note: $\mathbf{x} = \mathbf{x}^{(k)}$)
 $\mathbf{r} = \mathbf{r} - t\mathbf{u}$; (Note: $\mathbf{r} = \mathbf{r}^{(k)}$)
 $\mathbf{w} = C^{-1}\mathbf{r}$; (Note: $\mathbf{w} = \mathbf{w}^{(k)}$)
 $\beta = \sum_{j=1}^n w_j^2$. (Note: $\beta = \langle \mathbf{w}^{(k)}, \mathbf{w}^{(k)} \rangle$)

Step 6 If $|\beta| < TOL$ then
 if $\|\mathbf{r}\| < TOL$ then
 OUTPUT ('Solution vector'; $x_1, \dots, x_n$);
 OUTPUT ('with residual'; $r_1, \dots, r_n$);
 (The procedure was successful.)
 STOP

Step 7 Set $s = \beta/\alpha$; ($s = s_k$)
 $\mathbf{v} = C^{-t}\mathbf{w} + s\mathbf{v}$; (Note: $\mathbf{v} = \mathbf{v}^{(k+1)}$)
 $\alpha = \beta$; (Update α .)
 $k = k + 1$.

Step 8 If ($k > n$) then
 OUTPUT ('The maximum number of iterations was exceeded.');

SOR

To solve $A\mathbf{x} = \mathbf{b}$ given the parameter ω and an initial approximation $\mathbf{x}^{(0)}$:

INPUT the number of equations and unknowns n ; the entries a_{ij} , $1 \leq i, j \leq n$, of the matrix A ; the entries b_i , $1 \leq i \leq n$, of $\mathbf{b}$; the entries XO_i , $1 \leq i \leq n$, of $\mathbf{XO} = \mathbf{x}^{(0)}$; the parameter ω ; tolerance TOL ; maximum number of iterations N .

OUTPUT the approximate solution $x_1, \dots, x_n$ or a message that the number of iterations was exceeded.

Step 1 Set $k = 1$.

Step 2 While ($k \leq N$) do Steps 3–6.

Step 3 For $i = 1, \dots, n$

$$\text{set } x_i = (1 - \omega)XO_i + \frac{1}{a_{ii}} \left[\omega \left(- \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}XO_j + b_i \right) \right].$$

Step 4 If $\|\mathbf{x} - \mathbf{XO}\| < TOL$ then OUTPUT ($x_1, \dots, x_n$);
(The procedure was successful.)
STOP.

Step 5 Set $k = k + 1$.

Step 6 For $i = 1, \dots, n$ set $XO_i = x_i$.

Step 7 OUTPUT ('Maximum number of iterations exceeded');
(The procedure was successful.)
STOP.



GAUSS-SEIDEL

To solve $A\mathbf{x} = \mathbf{b}$ given an initial approximation $\mathbf{x}^{(0)}$:

INPUT the number of equations and unknowns n ; the entries a_{ij} , $1 \leq i, j \leq n$ of the matrix A ; the entries b_i , $1 \leq i \leq n$ of $\mathbf{b}$; the entries XO_i , $1 \leq i \leq n$ of $\mathbf{XO} = \mathbf{x}^{(0)}$; tolerance TOL ; maximum number of iterations N .

OUTPUT the approximate solution $x_1, \dots, x_n$ or a message that the number of iterations was exceeded.

Step 1 Set $k = 1$.

Step 2 While $(k \leq N)$ do Steps 3–6.

Step 3 For $i = 1, \dots, n$

$$\text{set } x_i = \frac{1}{a_{ii}} \left[- \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}XO_j + b_i \right].$$

Step 4 If $\|\mathbf{x} - \mathbf{XO}\| < TOL$ then OUTPUT $(x_1, \dots, x_n)$;
(The procedure was successful.)
STOP.

Step 5 Set $k = k + 1$.

Step 6 For $i = 1, \dots, n$ set $XO_i = x_i$.

Step 7 OUTPUT ('Maximum number of iterations exceeded');
(The procedure was successful.)
STOP.



JACOBI

To solve $A\mathbf{x} = \mathbf{b}$ given an initial approximation $\mathbf{x}^{(0)}$:

INPUT the number of equations and unknowns n ; the entries a_{ij} , $1 \leq i, j \leq n$ of the matrix A ; the entries b_i , $1 \leq i \leq n$ of $\mathbf{b}$; the entries XO_i , $1 \leq i \leq n$ of $\mathbf{XO} = \mathbf{x}^{(0)}$; tolerance TOL ; maximum number of iterations N .

OUTPUT the approximate solution $x_1, \dots, x_n$ or a message that the number of iterations was exceeded.

Step 1 Set $k = 1$.

Step 2 While $(k \leq N)$ do Steps 3–6.

Step 3 For $i = 1, \dots, n$

$$\text{set } x_i = \frac{1}{a_{ii}} \left[- \sum_{j=1, j \neq i}^n (a_{ij} XO_j) + b_i \right].$$

Step 4 If $\|\mathbf{x} - \mathbf{XO}\| < TOL$ then **OUTPUT** $(x_1, \dots, x_n)$;
(The procedure was successful.)
STOP.

Step 5 Set $k = k + 1$.

Step 6 For $i = 1, \dots, n$ set $XO_i = x_i$.

Step 7 **OUTPUT** ('Maximum number of iterations exceeded');
(The procedure was successful.)
STOP.

LAGRANGE INTERPOLATION

Input: Length of data n , data arrays x and y , and target x-value x_target .

Output: Target y-value y_target

STEP 1 Initialize the target y-value and define n as the length of the data values for x .

STEP 2 FOR i to N : DO

Let $p_i = 1$.

FOR j to N : DO

Construct the Lagrangian polynomial: $(p_i *= (x_target - x_data[j]) / (x_data[i] - x_data[j]))$

STEP 3 Build the interpolating polynomial $y_target += y_data[i] * p_i$

STEP 4 STOP.

BISECTION (BINARY SEARCH) METHOD

To find a solution to $f(x) = 0$ given the continuous function f on the interval $[a, b]$, where $f(a)$ and $f(b)$ have opposite signs:

INPUT endpoints a, b ; tolerance TOL ; maximum number of iterations N_0 .

OUTPUT approximate solution p or message of failure.

Step 1 Set $i = 1$;
 $FA = f(a)$.

Step 2 While $i \leq N_0$ do Steps 3–6.

Step 3 Set $p = a + (b - a)/2$; (*Compute p_i .*)
 $FP = f(p)$.

Step 4 If $FP = 0$ or $(b - a)/2 < TOL$ then
OUTPUT (p); (*Procedure completed successfully.*)
STOP.

Step 5 Set $i = i + 1$.

Step 6 If $FA \cdot FP > 0$ then set $a = p$; (*Compute a_i, b_i .*)
 $FA = FP$
else set $b = p$. (*FA is unchanged.*)

Step 7 OUTPUT ('Method failed after N_0 iterations, $N_0 =', N_0$);
(*The procedure was unsuccessful.*)
STOP.



NEWTON'S METHOD

To find a solution to $f(x) = 0$ given an initial approximation p_0 :

INPUT initial approximation p_0 ; tolerance TOL ; maximum number of iterations N_0 .

OUTPUT approximate solution p or message of failure.

Step 1 Set $i = 1$.

Step 2 While $i \leq N_0$ do Steps 3–6.

Step 3 Set $p = p_0 - f(p_0)/f'(p_0)$. (*Compute p_i .*)

Step 4 If $|p - p_0| < TOL$ then
 OUTPUT (p); (*The procedure was successful.*)
 STOP.

Step 5 Set $i = i + 1$.

Step 6 Set $p_0 = p$. (*Update p_0 .*)

Step 7 OUTPUT ('The method failed after N_0 iterations, $N_0 =$ ', N_0);
 (*The procedure was unsuccessful.*)
 STOP.

SECANT METHOD

To find a solution to $f(x) = 0$ given initial approximations p_0 and p_1 :

INPUT initial approximations p_0, p_1 ; tolerance TOL ; maximum number of iterations N_0 .

OUTPUT approximate solution p or message of failure.

Step 1 Set $i = 2$;

$$q_0 = f(p_0);$$

$$q_1 = f(p_1).$$

Step 2 While $i \leq N_0$ do Steps 3–6.

Step 3 Set $p = p_1 - q_1(p_1 - p_0)/(q_1 - q_0)$. (*Compute p_i .*)

Step 4 If $|p - p_1| < TOL$ then

OUTPUT (p); (*The procedure was successful.*)

STOP.

Step 5 Set $i = i + 1$.

Step 6 Set $p_0 = p_1$; (*Update p_0, q_0, p_1, q_1 .*)

$$q_0 = q_1;$$

$$p_1 = p;$$

$$q_1 = f(p).$$

Step 7 OUTPUT ('The method failed after N_0 iterations, $N_0 =$ ', N_0);

(*The procedure was unsuccessful.*)

STOP.

